A pairwise-comparison decision tree for sampling in DeLP3E

Depth 3 — 5-fold CV accuracy **0.846** — trained on 2,339 literal queries across 15 structural configurations

Decision rules

Each rule below corresponds to one branch of the tree. The condition between two features is always a strict order test: the rule's outcome depends only on which of the two structural quantities is larger, never on their absolute values. This makes every rule scale-invariant by construction.

R1 (**Worlds**, n=901, leaf purity 100%)
IF `af_avg_body_size` < `af_rule_to_var_ratio` AND `em_var` < `n_annots`, THEN sample by **Worlds**.

R2 (**Programs**, n=352, leaf purity 99%)
IF `af_avg_body_size` ≥ `af_rule_to_var_ratio` AND `am_n_arguments` < `em_entropy` AND `af_avg_body_size` ≥ `af_avg_var_usage`, THEN sample by **Programs**.

R3 (**Worlds**, n=930, leaf purity 72%)
IF `af_avg_body_size` ≥ `af_rule_to_var_ratio` AND `am_n_arguments` ≥ `em_entropy` AND `af_pct_annotated` < `am_attack_in_degree_gini`, THEN sample by **Worlds**.

R4 (**Programs**, n=148, leaf purity 68%)
IF the same conditions as **R3** hold except that `af_pct_annotated` ≥ `am_attack_in_degree_gini`, THEN sample by **Programs**.

The four remaining leaves of the tree cover a combined n=8 samples and are omitted as statistically insignificant.

Feature definitions

`af_avg_body_size` Average number of literals in the body of non-fact rules.

`af_rule_to_var_ratio` Number of annotated rules divided by the number of distinct BN variables actually referenced in any annotation.

`em_var` Number of random variables in the Bayesian network (EM component).

`n_annots` Number of program rules that carry a non-trivial annotation.

`am_n_arguments` Total number of arguments produced by the DeLP solver on the AM component.

`em_entropy` Joint entropy (in bits) of the Bayesian network.

`af_avg_var_usage` Average number of annotations that reference any given BN variable.

`af_pct_annotated` Fraction of program rules that carry a non-trivial annotation.

`am_attack_in_degree_gini` Gini coefficient of the in-degree distribution in the AM attack graph; high values mean attacks concentrate on a few arguments.

The 64 structural candidate features generate $\binom{64}{2}$=2016 binary comparisons; after dropping near-constant ones (variance below 0.05) the tree was fitted on the remaining 594. It selected automatically the 6 comparisons that appear above. No numeric threshold appears anywhere in the model: every decision relies only on the relative order between two structural properties of the DeLP3E model.

Heuristic W — a smarter Worlds sampler

Heuristic W refines the recommended sampling method for rules **R1** and **R3** (about 1,812 of the 2,339 queries in our experiments). Today, **Worlds** samples a possible world uniformly at random from the BN's joint state space, translates it into the corresponding DeLP program, and asks the solver about the queried literal. Uniform sampling treats every world as equally informative, but in practice two kinds of worlds dominate the convergence of the interval bounds: (i) the worlds that the BN itself considers likely, because their probability mass is what the interval is integrating, and (ii) the worlds that activate rules whose heads sit on *decisive* arguments in the dialectical analysis, because the warrant decision of the query hinges on those arguments. Heuristic W combines both signals into a single sampling distribution.

What the heuristic does

At setup time, the sampler runs the DeLP solver once on the full program to obtain its attack graph and records, for every argument, how many incoming attacks it has. Each annotated rule is associated with the argument whose head literal it concludes, so each rule inherits an *influence score* equal to the in-degree of that argument.

At sampling time, the algorithm draws a world directly from the BN (using the network's natural sampling routine), so the probability that a given world is chosen is exactly the world's BN probability. Then the world's influence score is computed by inspecting which annotated rules become active under the world's variable assignment and summing the influence scores of those rules. The world is finally accepted into the sample with a probability proportional to the product of its BN probability and its influence score, normalised across the candidate batch. Worlds with low BN probability or that activate only peripheral rules are seldom retained; worlds that are probable *and* activate rules concluding heavily attacked arguments are retained almost always.

A concrete example

Consider a DeLP3E model whose BN has 12 random variables and 4,096 possible worlds. The current sampler picks each world with probability 1/4096. Suppose the BN concentrates roughly 80% of its probability mass on a few hundred worlds. Under uniform sampling, almost all of the time budget is spent visiting worlds that contribute very little mass to the final interval, and the bounds tighten slowly.

Now suppose the AM contains 14 arguments and that two of them, call them a_7 and a_8, are each attacked by five other arguments. The annotated rules concluding a_7 or a_8 inherit influence score 5; all the others inherit influence score 0, 1 or 2. Heuristic W generates candidate worlds directly from the BN, so the first few candidates are almost certainly among the 80% high-probability worlds. Among those, the heuristic prefers the worlds that activate at least one annotation concluding a_7 or a_8, because those drive the value of the queried literal. As the sampler progresses, the interval bounds tighten faster: each accepted world brings more probability mass and more relevant argumentative information than a uniformly chosen one. Worlds that are very probable but activate only neutral rules are still sampled, just less often than under uniform.

Why this is well-suited to R1 and R3

Rule **R1** fires when there are many annotated rules relative to BN variables and when the program space is larger than the world space. In this regime each world activates several annotations on average, so the influence score is informative even at small sample sizes. Rule **R3** fires when the attack graph already has a strong concentration of incoming attacks on a few arguments (the Gini coefficient is large), so the influence score gives a particularly sharp signal about which worlds are decisive. Both regions therefore benefit from the same combined criterion: respect the BN's own measure of likelihood, and prioritise the worlds whose argumentative consequences matter most.

Heuristic P — a smarter Programs sampler

Heuristic P refines the recommended sampling method for rules **R2** and **R4** (about 500 of the 2,339 queries). The current **Programs** sampler picks each annotated rule independently with probability one half, producing a uniformly random subset of the annotation set as the candidate program. This is convenient but wasteful: many of the candidates produced this way turn out to be inconsistent or unrepresentative of the BN, and each one still incurs the cost of a full DeLP solver call. Heuristic P replaces the uniform draw with a Bayesian-informed draw and adds a cheap pre-filter that discards obviously contradictory programs before the solver is invoked.

What the heuristic does

For each annotated rule, the heuristic first asks the BN how likely that rule's annotation formula is to evaluate to True. This is a marginal probability that can be computed once at setup time, since annotations do not depend on the choice of query. At sampling time, the rule is included in the candidate program with probability equal to that marginal. Rules whose annotation is almost always satisfied are present in nearly every sample; rules whose annotation rarely holds appear only occasionally; rules with intermediate marginals are present roughly that fraction of the time. In short, the distribution over candidate programs concentrates on the programs that the BN itself considers most representative of its joint distribution.

Once a candidate program has been drawn, Heuristic P performs a fast syntactic check before passing it to the DeLP solver. If the candidate contains a pair of strict rules that contradict each other — for instance the same literal appears as a strict fact and its complement also appears as a strict fact — the candidate is discarded and a new one is drawn. The check is purely textual, requires no SAT solving, and rejects in a few microseconds the candidates that would otherwise have consumed an entire solver call only to be reported as inconsistent. The result is that the time budget is spent almost exclusively on programs whose answer carries real information about the warrant interval.

A concrete example

Consider a DeLP3E model with 13 annotated rules whose annotations involve a small set of BN variables. The current sampler draws candidate programs uniformly from the 2^{13}=8192 possible subsets. In a typical run, a substantial fraction of those candidates contain strict-rule contradictions inherited from a randomly chosen combination of annotations; the solver wastes its time budget recognising the same contradictions over and over.

Heuristic P first inspects the BN. Suppose three rules have annotation marginals near 0.9, six have marginals close to 0.5, and four have marginals below 0.1. Drawing under the heuristic produces candidate programs in which the three high-probability rules are almost always present, the six mid-probability rules vary, and the four rare rules are usually absent. This already excludes most of the structurally implausible programs. The pre-filter then catches the residual contradictions: for example, if a high-probability fact a ends up alongside a mid-probability strict rule that derives $\neg a$, the candidate is discarded without any solver call. Over a fixed time budget, the heuristic visits many more informative programs than the uniform baseline, and the warrant bounds therefore tighten faster.

Why this is well-suited to R2 and R4

Rule **R2** fires when the AM is small relative to the entropy of the BN, which means that the small number of arguments respond strongly to which annotations are active; biasing the draw towards BN-plausible annotation subsets puts the sampler on the right region of the program space from the first iteration. Rule **R4** fires when the annotations are very dense, so the uniform sampler is especially wasteful and the pre-filter eliminates a large fraction of the contradictory candidates. In both cases the heuristic concentrates the limited solver budget on the candidates that genuinely move the interval bounds.

Both heuristics fall back to the current uniform strategy if any of their inputs is unavailable (for instance, if the attack graph cannot be obtained for a very large AM, or if the BN marginals are not computed within the setup budget). The fallback preserves the soundness of the underlying samplers; the heuristics only ever change which candidates are tried first, never which candidates are admissible.